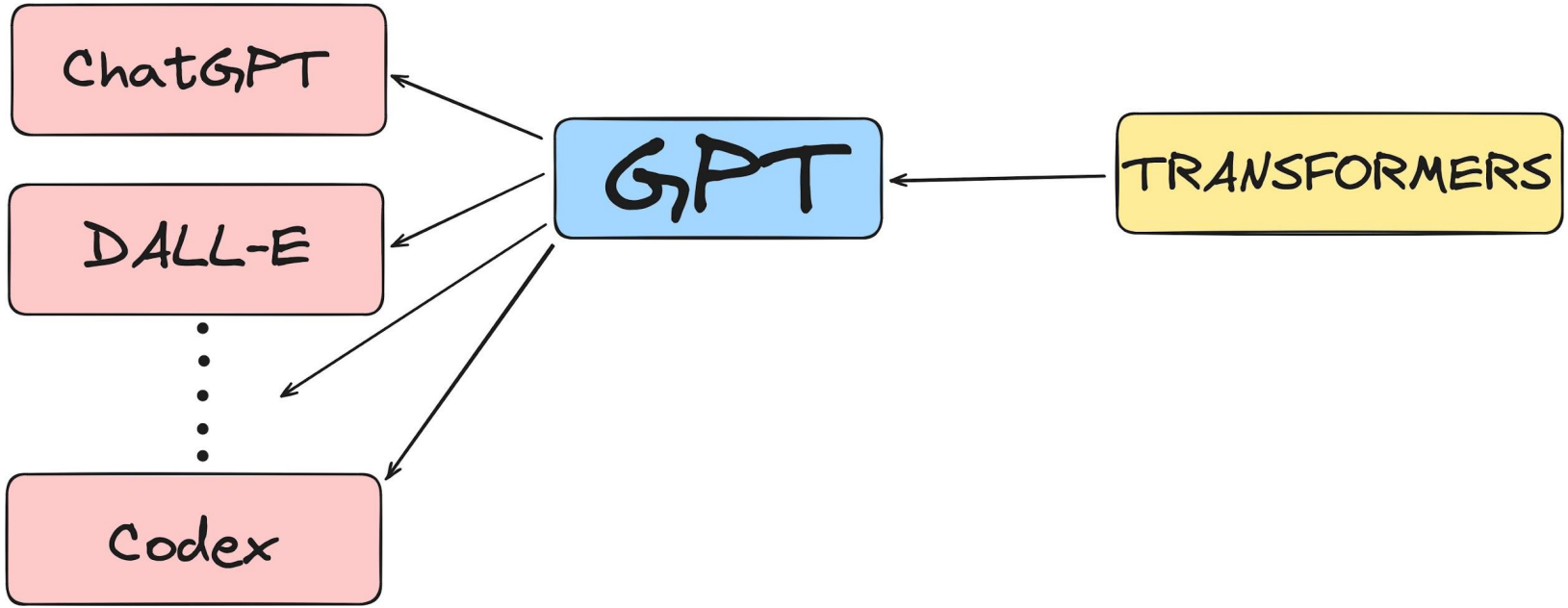

ChatGPT
DALL-E
Codex
GPT
TRANSFORMERS

What is "GPT"?

Generative

- Produces output like text (including code), image, and voice

Pre-trained

- Already trained on large corpuses including millions of webpages, books etc

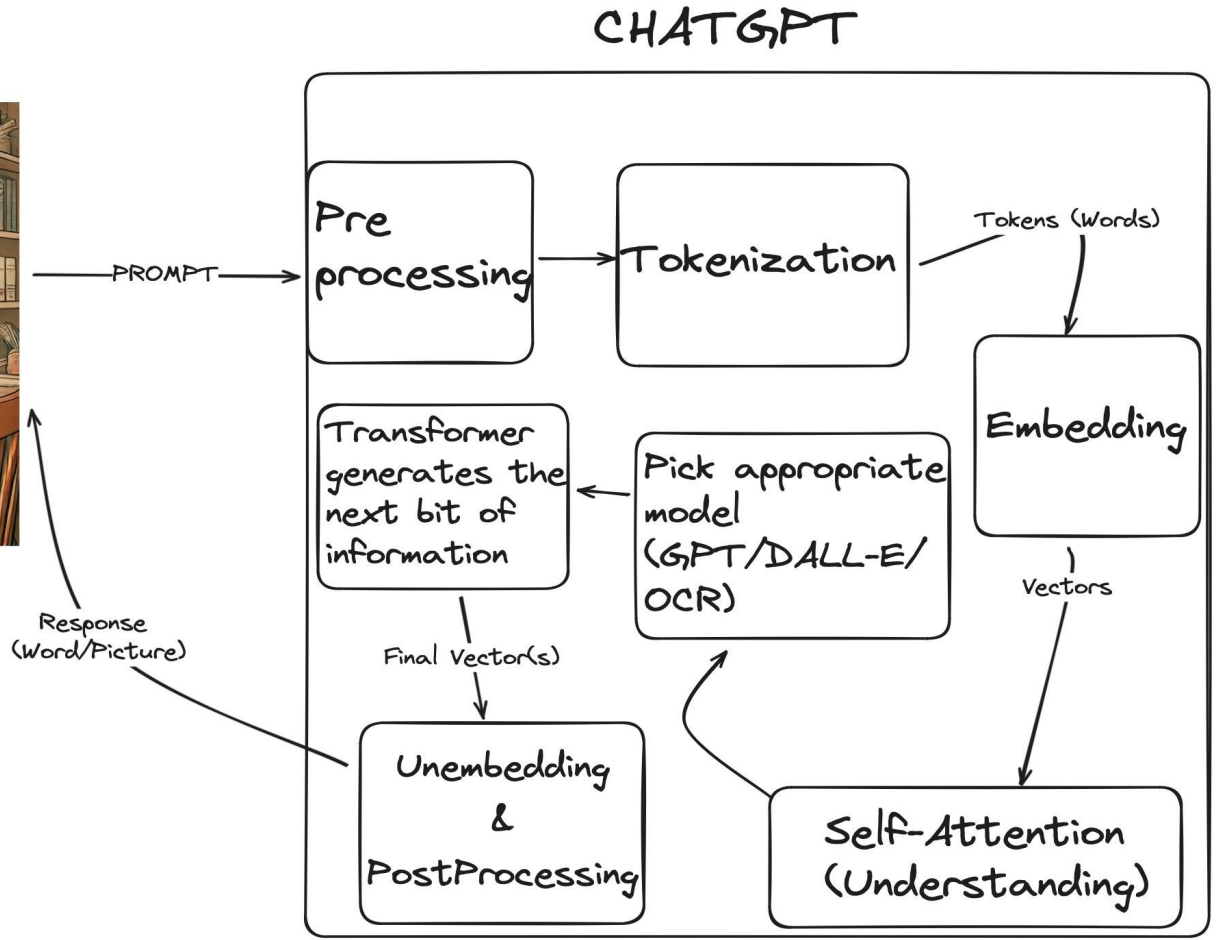
  GPT3 was trained on all of Wikipedia amongst other things
- Leaves room for additional training (fine-tuning) for specific tasks like classifying news articles into categories

Transformer

(attention is all you need)

- The core neural network architecture that every generative model is built on
- Iteratively calculates the probability of what the next piece of information might be based on current inputs like the next note in Beethoven's 5th Symphony or the next word in Fee Fi Fo

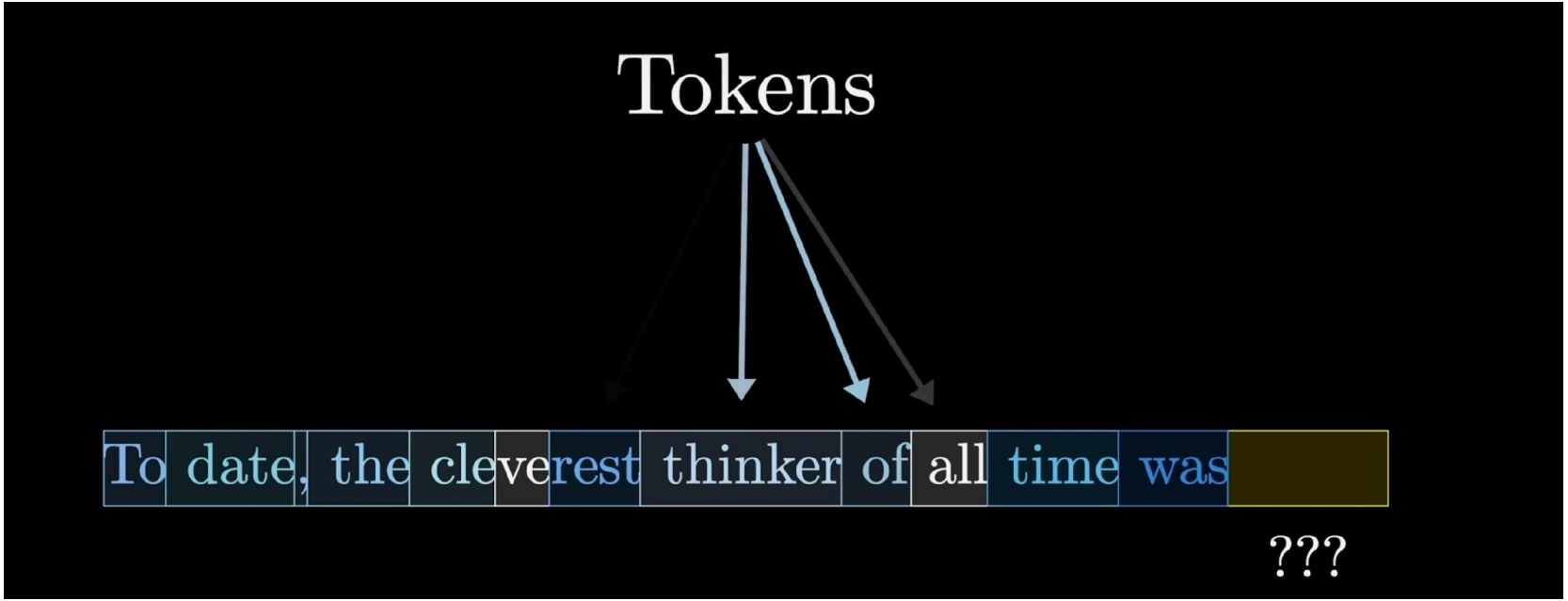
ChatGPT


CHATGPT
PROMPT
Pre processing
Tokenization
Tokens (Words)
Embedding
Vectors
Self-Attention (Understanding)
Pick appropriate model (GPT/DALL-E/ OCR)
Transformer generates the next bit of information
Final Vector(s)
Unembedding & PostProcessing
Response (Word/Picture)

What happens to a “cleaned” Input?

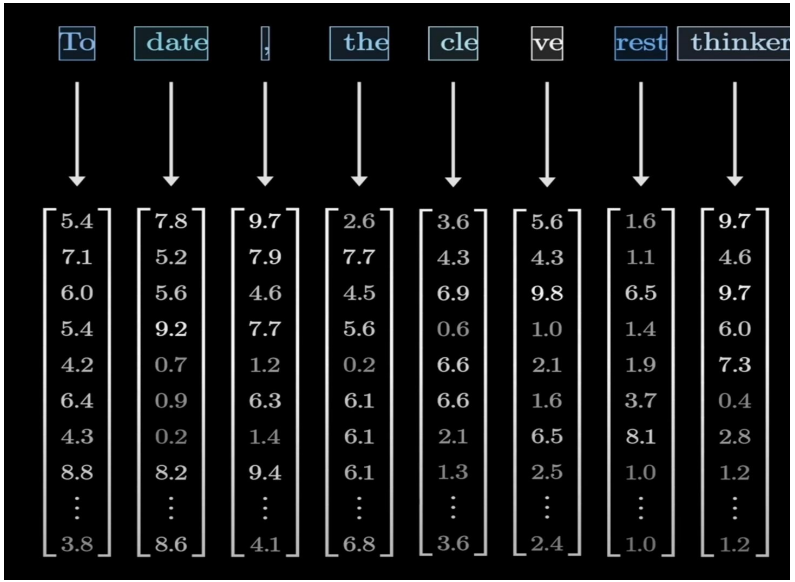
Tokenizing

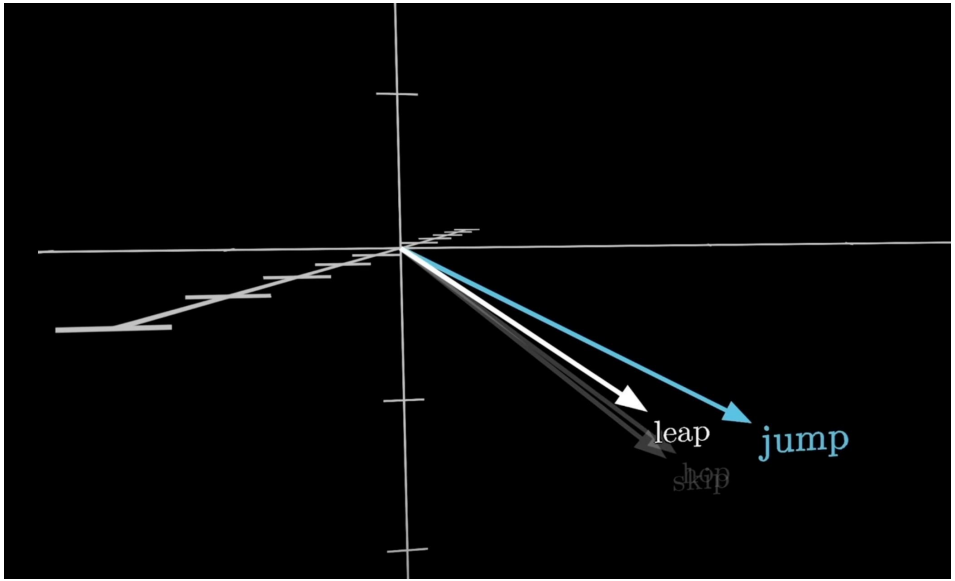
- Breaking inputs into chunks
- Words/characters for text, sound snippets for voice, pixels groups for image



Embedding

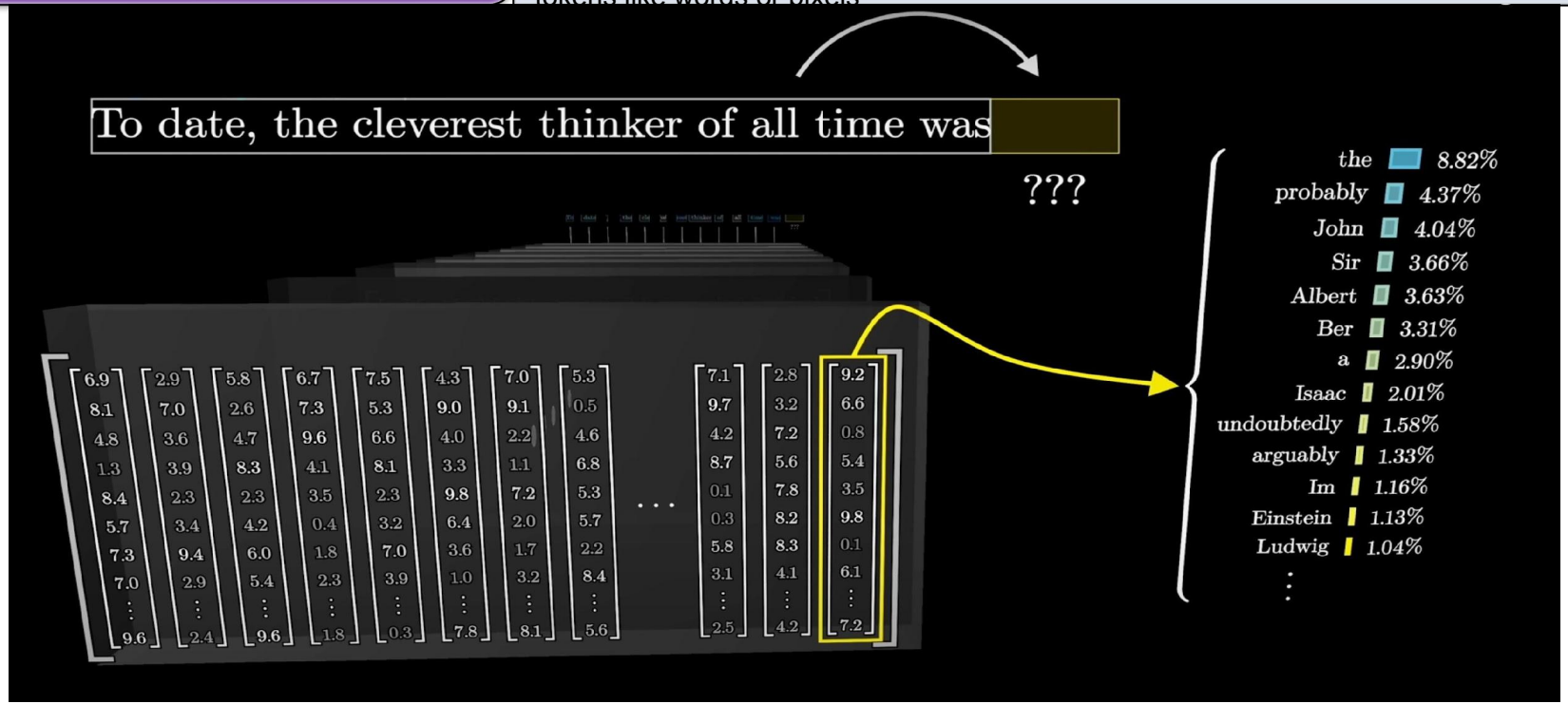
- Encoding chunks into vectors in an N-dimensional vector space
- Similar chunks end up as vectors close to each other (words with similar meaning, pixels that have similar RGB values)
- Can do vector operations like add, subtract, dot, and cross products
- [Vector for “King”] – [Vector for “Man”] + [Vector for “Woman”] = [Vector for “Queen”]





Unembedding

- After many rounds of Attention and MLPs, a final vector is produced which is the official next bit of information
- Now we reverse the embedding into a useful token that is the same as the input tokens like words or pixels



“Clean Inputs” – TEXT - Stop words

- Human languages, like English, have a lot of filler words that are grammatical but don’t add any meaning to the context, these are called stop words

- Some examples in English are “a”, “the”, “an”

- Removing these stop words will lower tokens used and will help you get a more bang for your buck when fine-tuning a Large Language Model

- Utilities to remove them: NLTK, a python package, is the most used way to remove them, the full list of stop words can be found here.

“Clean Inputs” – TEXT - Stemming

- A way to remove prefixes and suffixes from words to distill meaning and “stem” each word to its basic forms
- Doing this standardizes words and allows models to be sure of each word’s meaning
- Simple examples are: “walks” becomes “walk”, “retrieval” becomes “retrieve”
- Done using algorithms that, thankfully, we don’t have to develop
- The most common one is PorterStemmer, which uses rules such as removing common suffixes like “ed”, “ing”, converting plurals to singulars (cars to car), and reducing adjectives to their base forms (happier to happy)

“Clean Inputs” – TEXT – Stemming CONTD.

- Stemming is useful in information retrieval, search, and data mining
- Use it wisely! Be cautious of ”over stemming” – All stemmers have false positives where a word might be reduced to a form that is meaningless, like “university” might be reduced to “univers”
- Stemming words can make them lose their contextual meaning e.g. agreement -> agree
- Compound words are poorly handled (e.g “Whiteboard”)
- Stemmers usually don’t handle proper nouns like names well.

“Clean inputs” – Text - Lemmatization

- More sophisticated than stemming: better contextual awareness
- Reduces words to their dictionary basic form (lemma) so ”university” will remain as such
- “Running” and “ran” will both be reduced to “run”
- This helps models group certain words (and sentences) together
- Helps reduce overhead and reduce dimensionality of vectors that words produce
- Helps info retrieval: “Best coffee” will also retrieve results for “good coffee”
- Done using NLTK as well, (WordNetLemmatizer) (code demo)

“Clean inputs” – Text – Lemmatization CONTD.

- Beware! Lemmatization is not king.
- Context can still be lost. E.g. “running" and "run" have different nuances in certain contexts, which might be lost after lemmatization.
- It is computationally expensive and slow because of the complex linguistic analysis
- Rules are specific to each language, requiring different approaches and tools for different languages.
- Ambiguous words can lead to incorrect lemmatization. For example, "bats" can be the plural of "bat" (the animal) or a form of the verb "to bat".
- Different lemmatization tools might produce inconsistent results for the same text. (so use the same tool please)

STEMMING & LEMMATIZATION DEMO

What is RAG?

Hybrid Approach: Combines LLMs with information retrieval systems to enhance response accuracy and relevance.

Why RAG?

Language Models (LMs) like GPT-4 are powerful for generating human-like text but face key limitations in certain tasks.

Challenges with LLMs:

- **Static Knowledge:** LLMs are limited to knowledge up to their last training cut-off and lack real-time updates.
- **Contextual Limits:** They struggle with generating accurate responses for highly specific or less common topics without sufficient context.
- **Large Scale Data Handling:** Handling vast amounts of information and ensuring relevance and accuracy in responses can be challenging.

How RAG Works?

Retrieval Component: Fetches relevant documents or data from a knowledge base or database.

Generation Component: LLM generates responses based on both retrieved information and its own capabilities.

Benefits of RAG

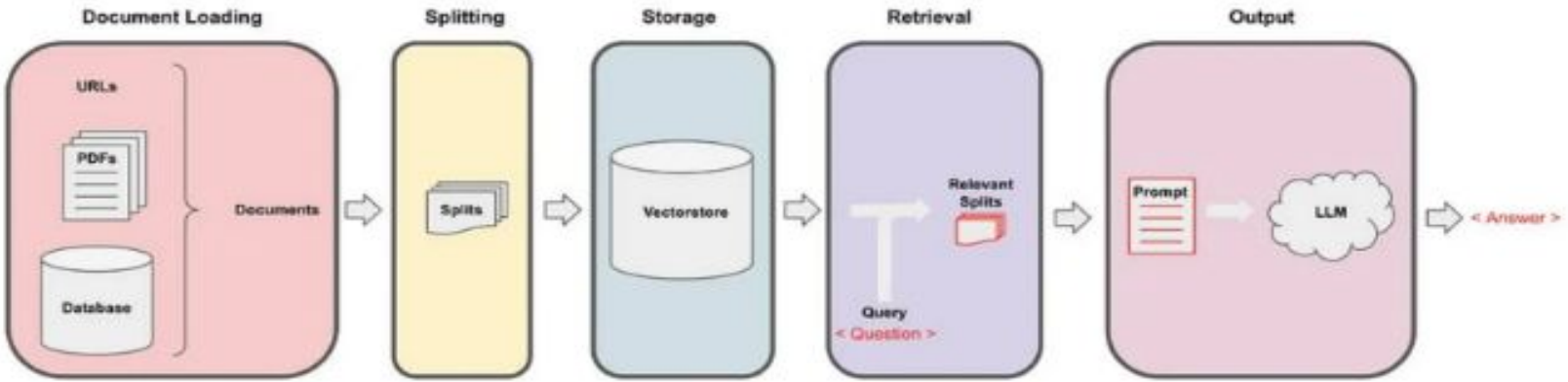
Updated Knowledge: Access to real-time or recent information.

Enhanced Accuracy: More precise and contextually relevant answers.

Scalability: Handles large-scale data more effectively, improving response quality.

RAG - 101

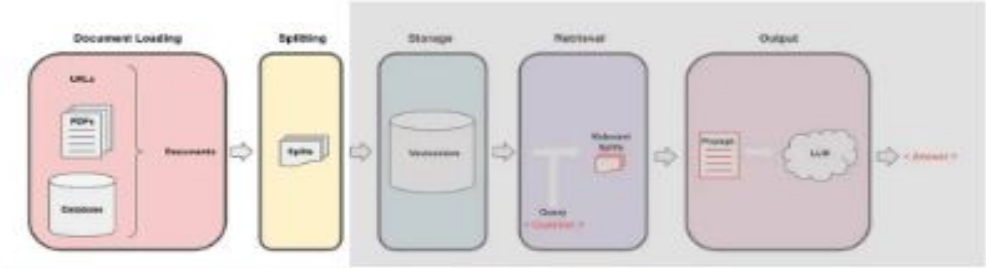
Search & Summarize In 4 Steps



RAG

Step 1 - Document loading

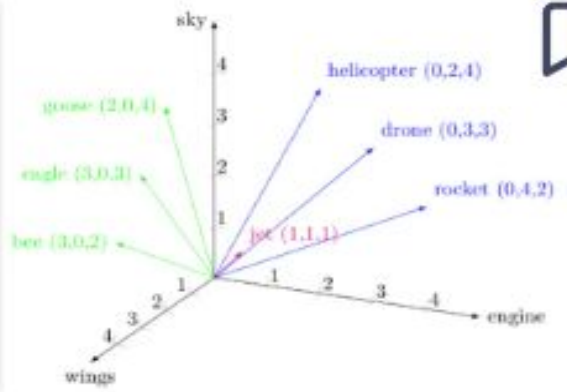
- Documents are loaded from data connectors
- They are split into chunks

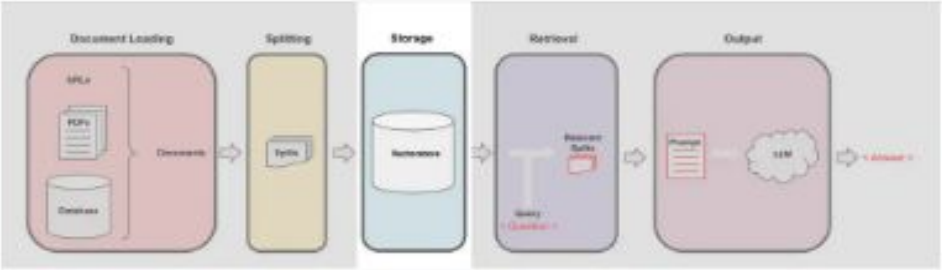


RAG

Step 2 - Embeddings

- Chunks are 'transformed' into vectors (numbers)
  - ✓ It's the process of word embedding, using a pre-trained model
  - ✓ hundreds (even thousands !) of dimensions are required to represent the space of all words
- Vectors are stored in a dedicated database (a vector database)

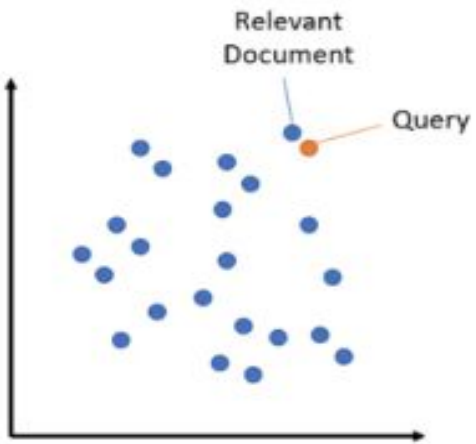


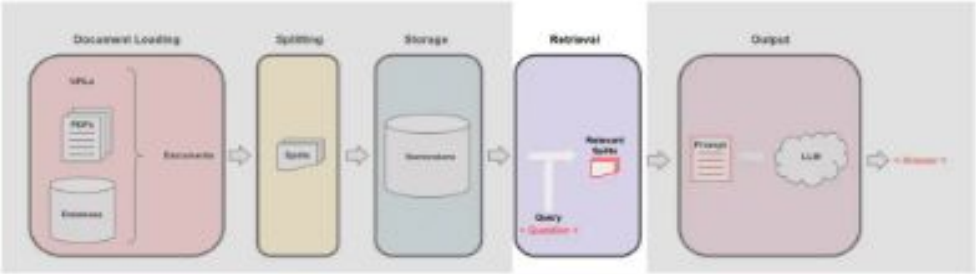


RAG

Step 3 - Retrieval

- Previous steps were preparatory work, now comes the live part
- Question is vectorized as well, used as an input for similarity search
- Most relevant chunks are retrieved, i.e. vectors coordinates are close together



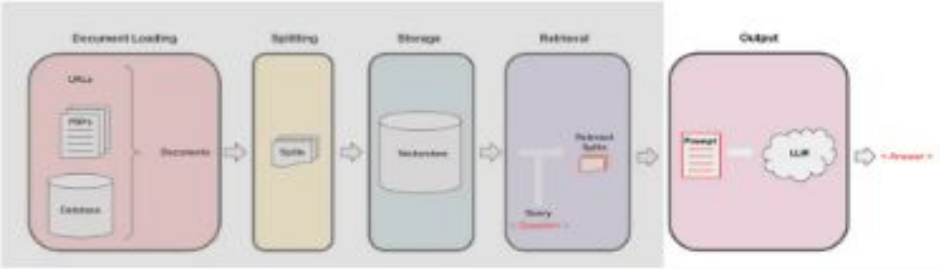


RAG

Step 4 - Generation

- Retrieved chunks are used to feed the LLM prompt context
- Question is added to the prompt
- LLM reads the prompt and generates a natural language answer
- During this inference time, the model requires a lot of GPU power !

```
prompt =
"""\

Context information is below.

--------------------- {context_str} ---------------------
Given the context information and not prior knowledge, answer the query
asking about citations over different topics. Please provide your answer in
the form of a structured JSON format containing a list of authors as the
citations. Some examples are given below.

{few_shot_examples}

Query: {query_str}

Answer: \

"""
```



Types of RAG

- Simple RAG
- Simple RAG with Memory
- Branched RAG
- Adaptive RAG
- Corrective RAG (CRAG)
- Self-RAG
- Agentic RAG